



OS Development

6. Scheduling

Prof. Dr. Michael Schöttner
Dr. Fabian Ruhland

- Motivation & Definitions

- Thread Switching

- Scheduling

- Scheduler in HeineOS

- Summary

- Goal: Run application as independent threads
 - We want a dynamic (not static!) schedule
 - Threads can either yield the CPU voluntarily, or the scheduler withdraws it
- Solution: **Scheduler** in the operating system
 - Decides which thread runs next
 - If a thread blocks, the scheduler automatically switches to the next one

- **Cooperative Multithreading:**

Threads must yield the CPU voluntarily, as the operating system cannot withdraw it

- **Preemptive Multithreading:**

Threads can yield the CPU voluntarily, but the operating system is also able to withdraw forcefully

- **Address Space:** Accessible memory at a given point in time
- **Process:** Address space, thread(s) and metadata (program that is currently being executed and its data)
- **Thread:** Independent activity
 - Always tied to a process
→ shares data, code and resources with threads of the same process
 - Separate register set and stack
 - (Sometimes called *lightweight process*)

- Motivation & Definitions

- Thread Switching

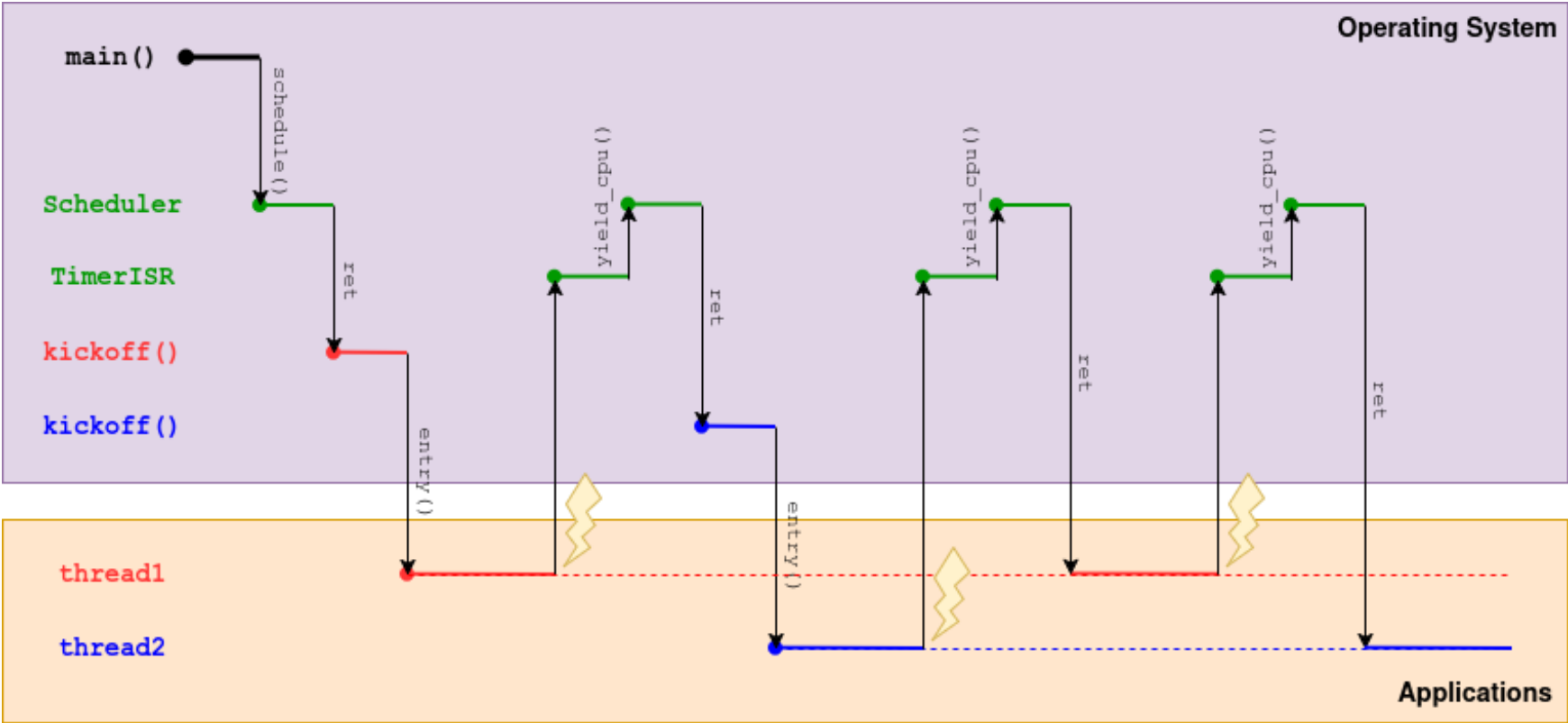
- Scheduling

- Scheduler in HeineOS

- Summary



- Problem: How can the operating system withdraw the CPU from a thread?
- Solution: Use the timer interrupt (PIT)
 - Timer trigger periodic interrupts (e.g., every 1 ms)
 - Timer interval is programmable
 - Every timer interrupt is a *tick* → Switch threads after a certain amount of ticks (e.g. 10)



- Motivation & Definitions
- Thread Switching
- Scheduling
- Scheduler in HeineOS
- Summary

- Usually, many threads wait for the CPU
- Deciding which thread is next must be quick
- Scheduler manages all waiting threads in a *Ready Queue*
 - In practice, multiple queues (for different priorities) or trees are used
- Blocked threads are not stored in the ready, as always skipping them causes overhead

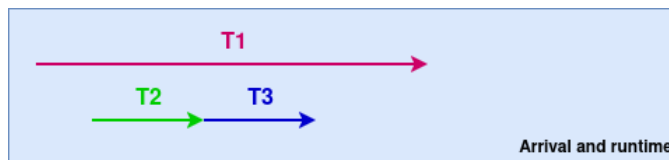
- Fairness: Each thread should get (on average) the same amount of CPU time
- Waiting time: Minimal time spent in ready queue
- Throughput: Maximum number of threads per time unit
- ...

First Come First Served (FCFS)

- Threads are processed in the same order they arrive in the ready queue
- CPU ownership until end or voluntary yield
- Example: Good case



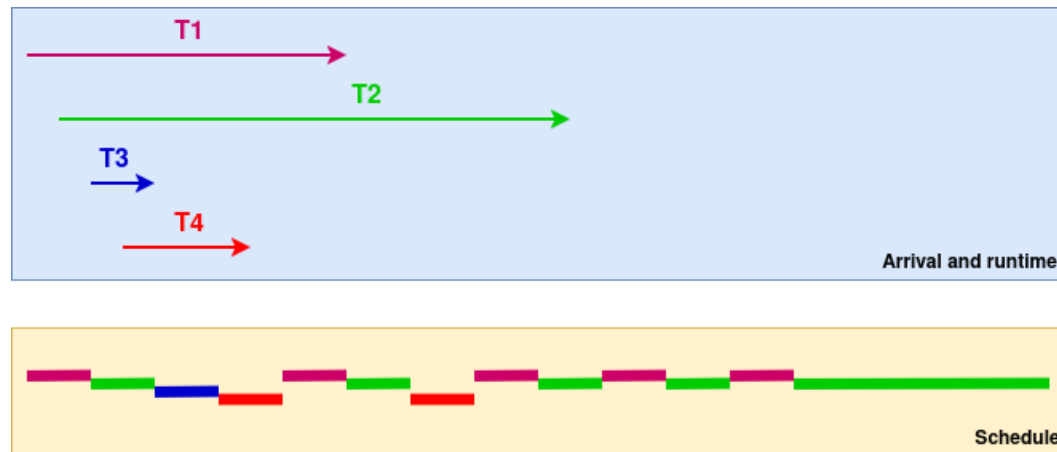
- Example: Worst case



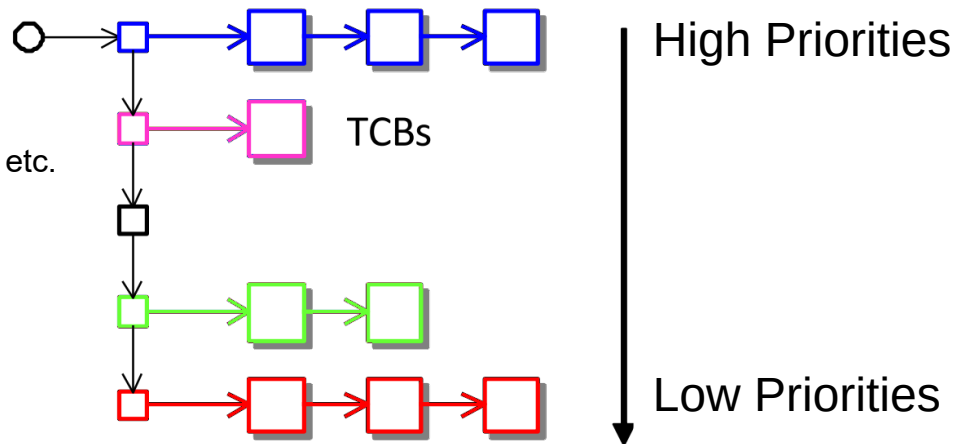
- Disadvantage:
 - Convoy effect = Short threads bank up behind a long thread

Round Robin (RR)

- Goal: Equal division of CPU time
- Common strategy:
 - Threads are processed in order of their arrival
 - Current thread is preempted after a fixed time slice (e.g., 10-100 ms) and enqueued at the end of the ready queue
- Problem: Right choice of time slice
 - Reasonable proportion between time slice and time needed for context switch
 - Large time slices reduce context switches, but increase latency



- It is desirable to distinguish threads by their importance
→ Assign a priority number to each thread
- Scheduler prefers threads with higher priorities
- Implementation:
 - One ready queue per priority
 - Allows fast retrieval of right thread
 - TCB = Thread Control Block
 - Information about the thread: ID, Stack, etc.



- Feedback → Dynamic adaption of scheduling criteria, depending on thread behaviour (e.g., CPU usage, sleep time)
- Adaptation is possible via priority, ready queue insertion (beginning or end) and time slice length
- Two fundamental strategies:
 - Bump up priority of waiting threads
 - Downgrade priority of CPU intensive threads

- If all threads are waiting, the CPU is free and an idle thread runs:
 - May not stop, has lowest priority and is interruptible at any time
 - Busy looping can be avoided using special instructions
 - Example: „HLT“ (x86): stops CPU; (Timer-) interrupt wakes cpu up again

- Idle times can also be used for:
 - Defragment filesystems
 - Garbage collection
 - Software updates
 - ...

- Arbitrary assignment of threads to CPU cores is bad:
 - If thread t has lastly used core p , there is a chance that parts of its address space are still inside the cache/TLB of p
 - (If another thread of the same address space was the last active one on this core)
- Scheduler wants to achieve an even load across all CPU cores
- Core affinity allows defining preferences for specific cores
 - Can weaken priorities

- Motivation & Definitions
- Thread Switching
- Scheduling
- Scheduler in HeineOS
- Summary

- Cooperative FCFS scheduler
- Single ready queue without priorities
- Current thread is not in ready queue, managed in `SchedulerState` struct
- Next thread to run is always the one at the ready queue's head
- Threads that have just run or freshly arrive are always enqueued at the ready queue's tail

- Motivation & Definitions
- Thread Switching
- Scheduling
- Scheduler in HeineOS
- Summary

- Threads allow concurrency in applications
- If one thread is blocked, another can run on the
- Scheduler decides which thread runs next
- In preemptive multithreading, threads are forcefully switched using the timer interrupt